

Interaction FITs the Trend: An Enhanced Two-Tower Approach to Social Media Feed Ranking at Scale

Max Luis Rodriguez

March 2026

Abstract

With the rise of mass social media use globally, it is increasingly important to study the recommender systems that determine which content users see and engage with. This report documents my research contributions to the Green Earth Social / Social Media Lab engagement-prediction training and evaluation pipeline for Bluesky user and media data. The current system implements a five-stage workflow: data acquisition, target-post construction, user-history construction, model training, and modular evaluation. Within this pipeline, I study four trainable recommender families: a pointwise multilayer perceptron (MLP), a standard two-tower model, a FIT-augmented two-tower model (with an additional late-interaction ablation), and a collaborative-filtering baseline. The findings of this report support the viability of enhanced early-late interaction two-tower (FIT) models for social media engagement prediction, achieving performance comparable to a higher-latency MLP baseline. More broadly, the report situates these results within the larger systems problem of candidate generation, seen- and unseen-user generalization, and content bias, all of which are critical downstream considerations for deployment-ready social media recommender systems.

1 Introduction

In modern social media systems, feed-ranking algorithms shape which voices are amplified and which content patterns are reinforced across millions of users, making their design a pressing issue of both social responsibility and technical implementation [11]. This report describes my engineering work and experiments on a multi-stage engagement-prediction pipeline designed to predict whether a user will engage with (like) a candidate post. The broader motivation is not only predictive performance, but also analyzing trade-offs between different ranking architectures in terms of both quantitative and qualitative performance.

I train four different engagement-prediction recommender systems with distinct underlying architectures on Bluesky posts, using post embeddings and user like histories. Training a robust engagement-based feed-ranking model serves as a prerequisite for our broader research goals, which include analyzing downstream effects such as the extent to which engagement-optimized systems may promote sensationalized, polarizing, or toxic content.

Through collaboration with the Green Earth Social Berkeley group under Senior UC Berkeley Scientist Jonathan Stray, we have the opportunity to evaluate our media algorithm at industry scale (i.e., for Skylight Social). This setting introduces challenges in designing effective, low-latency, and cost-efficient user feed management systems. To address these, we adopt and evaluate a fully interacted two-tower recommendation approach [13]. To illustrate our motivation, let ℓ denote the number of candidate posts and N the cost of one full-context user-item pair engagement prediction scoring pass (e.g., using an MLP). Exhaustive ranking then scales as $O(\ell N)$. By leveraging the efficiency of the two-tower architecture, we reduce the candidate set before applying a more expressive scorer. This brings inference closer to $O(N + \ell M)$, where $M \ll N$ is the cost of computing a lightweight similarity score against precomputed item embeddings [13].

The core research questions of this report are: *Can we effectively predict user engagement using a lightweight, fully interacted two-tower recommender system, even under realistic cold-start conditions? How should performance be interpreted across architectures when negatives are sampled non-likes rather than verified dislikes? And to what extent can we track the promotion of toxic or sensationalized content by our models?* To address these questions, we conduct preliminary qualitative analyses of content and sentiment, alongside comprehensive quantitative evaluations against full user-item interaction MLP baselines and a collaborative-filtering baseline trained on observed likes.

2 Related Work

Black-box social media feed-ranking algorithms: Algorithms such as Twitter were previously open source, relying on reverse-chronological, non-engagement-based personalization [8]. In contrast, Meta (Instagram and Facebook) algorithms are not open source and instead use machine learning to predict what users will find most valuable and relevant [3]. Other major platforms, such as YouTube, incorporate user preferences and behavior into their machine learning-based feed-ranking systems [14]. More recently, X released its recommendation algorithm [12], providing evidence for the widespread use of deep learning in modern social media ranking systems. Through our research, we aim to take these algorithms out of the black box by providing a fully open-source pipeline to study the effects of engagement-prediction algorithms.

Lightweight social media feed ranking: Inference-time latency plays a critical role in social media feed ranking. Pre-ranking is widely adopted to improve efficiency and scalability for large user bases [13, 10]. The two-tower recommender system is one of the most widely used approaches [10, 12], consisting of separate learnable towers for user and item embeddings. This approach typically uses a dot-product scoring function to bring user embeddings closer to the embeddings of liked items through backpropagation [15]. Our experiments build on this framework by comparing a standard two-tower baseline with a variant that introduces enhanced early and late interaction between the towers [13].

Implicit feed ranking: Collaborative filtering remains a foundational baseline for recom-

mender systems. Matrix-factorization methods represent users and items through latent factors and bias terms, providing a compact and interpretable representation of preference structure [2]. More recent work continues to use collaborative filtering as a lightweight baseline across domains [4]. In this repository, collaborative filtering serves an additional purpose: it can be trained using only observed positive interactions, making it a useful comparison to classifiers trained on explicit positive and randomly sampled negative pairs.

3 Approach

Main task. The repository implements a five-stage pipeline:

1. Stage 1 (`get_data`) loads likes and posts, filters users/posts, and writes post embeddings.
2. Stage 2 (`target_posts`) constructs target rows and assigns train/validation/holdout splits.
3. Stage 3 (`user_history`) builds per-target-row prior-like user histories.
4. Stage 4 trains one of several recommender models.
5. Stage 5 evaluates holdout predictions with modular analyses, including cold-start curves and performance-inequality summaries.

3.1 Dataset

For the experiments and results described in this report, I use Bluesky data collected throughout January 2026. In Stage 1, inactive users are filtered out by requiring at least two liked posts, and very active users are capped at a maximum of 100 user-item like events. Stage 1 produces 169,793 likes and 144,716 embedded posts using the `all_MiniLM_L6_v2` sentence-transformer model, yielding 384-dimensional post vectors. Stage 2 then creates 169,787 target rows across 8,214 users. The split sizes are 70,306 train rows, 30,786 validation rows, 35,418 `holdout_seen_users` rows, and 33,277 `holdout_unseen_users` rows. The `holdout_seen_users` split used for collaborative-filtering tests contains 4,773 users. The unseen-user holdout consists of 1,637 users selected via a deterministic hash-based split with a configured holdout-user fraction of 0.2. Stage 3 attaches prior-like histories to each target row: 161,561 rows have non-empty history, while 8,226 have empty history. The mean history length is 19.9 likes, the median is 16, and the maximum is 84.

We make an important distinction between target posts and user-history embedding posts. For a target row at time t , the positive candidate post is the post liked at time t , and the negative candidate is one sampled non-liked post from the same time bucket. The user-history representation for that row is built only from likes strictly earlier than t (with an additional, optional history buffer). With this temporal data structuring approach, the current target post is never included in its own user history, and future likes are never visible. The user representation is recomputed separately for each target row as a temporally valid snapshot of the user’s past behavior (Figure 1).

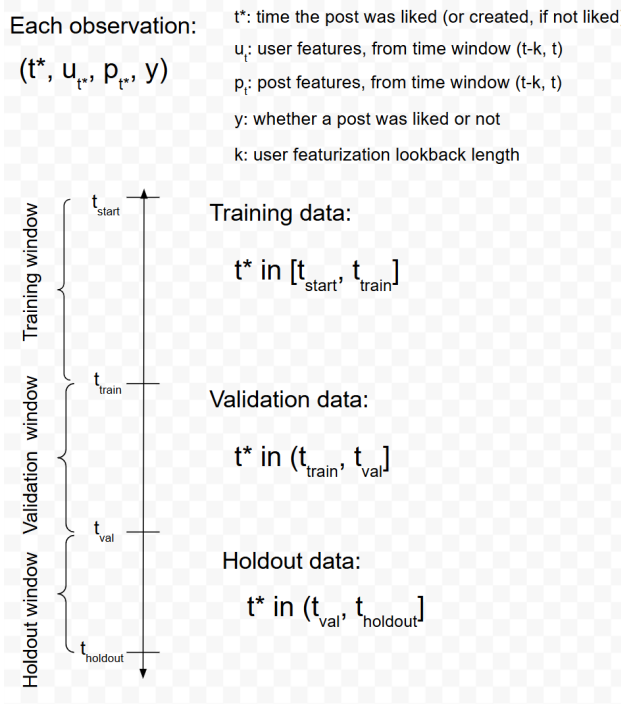


Figure 1: Temporal data split visualization illustrating the construction of user histories and target posts, created by Green Earth Social ML Engineer David Freifeld.

3.1.1 User History Encoders

The user history encoding differs by model type. For the MLP, user history is collapsed into a single fixed vector using a deterministic summarization rule (i.e., mean pooling via exponential moving average (EMA)). For our sequence encoder models such as `full_transformer` and `cross_attention`, prior liked posts are retained as an ordered sequence of post embeddings (most recent first), padded and masked, and the model learns a user representation from this sequence. We experiment across these encoding methods during FIT and standard two-tower training sweeps. For neural models, user-history encoding is part of the model and is applied during the forward pass.

Let a user’s prior liked-post embeddings for one target row be

$$H = [h_1, \dots, h_T], \quad h_t \in \mathbb{R}^D,$$

ordered most-recent-first, with mask

$$m_t \in \{0, 1\}.$$

Before either neural encoder is applied, the history embeddings are projected into the model’s hidden space and augmented with positional information:

$$x_t = \text{Proj}(h_t) + p_t,$$

where

$$\text{Proj} : \mathbb{R}^D \rightarrow \mathbb{R}^d$$

is a learned projection and p_t is a learned positional embedding.

Full transformer encoder. The `full_transformer` encoder first contextualizes the history using self-attention:

$$z_1, \dots, z_T = \text{Transformer}(x_1, \dots, x_T).$$

It then forms two pooled summaries of the contextualized sequence:

$$u_{\text{attn}} = \sum_{t=1}^T \alpha_t z_t, \quad \alpha_t = \text{softmax}(q^\top z_t),$$

and

$$u_{\text{mean}} = \frac{\sum_{t=1}^T m_t z_t}{\sum_{t=1}^T m_t}.$$

These are concatenated and projected to the final user representation:

$$u = \text{OutProj}([u_{\text{attn}} \parallel u_{\text{mean}}]).$$

Cross-attention encoder. The `cross_attention` encoder is a lighter alternative. It skips self-attention over the history and instead pools directly from the projected sequence:

$$u_{\text{attn}} = \sum_{t=1}^T \alpha_t x_t, \quad \alpha_t = \text{softmax}(q^\top x_t),$$

$$u_{\text{mean}} = \frac{\sum_{t=1}^T m_t x_t}{\sum_{t=1}^T m_t},$$

$$u = \text{OutProj}([u_{\text{attn}} \parallel u_{\text{mean}}]).$$

Collaborative-filtering data structure. For the collaborative-filtering baseline, input data is structured differently. The model does not use the 384-dimensional post embeddings directly as features. Instead, users are mapped to integer user IDs and posts to integer item IDs, and training is performed over an implicit user–item interaction matrix. Positive interactions are constructed from observed likes in the training split. Unobserved user–item pairs are used during optimization as implicit negatives.

3.2 Two-Tower

Design. The standard two-tower model learns a user representation and a post representation in a shared latent space. Let $\mathbf{u}$ denote the user vector and $\mathbf{v}$ denote the post vector. The model

predicts engagement as

$$p(y = 1 \mid \text{user}, \text{post}) = \sigma(\mathbf{u}^\top \mathbf{v}),$$

where $\sigma(\cdot)$ is the sigmoid function.

The code supports three user encoders for two-tower training. The simplest is a summarized-history mode, in which the user vector is precomputed by a deterministic mean-pooling history summarizer. The two trainable sequence encoders are the full transformer and lighter cross-attention pooling encoders. The transformer variant offers the highest modeling capacity, whereas the cross-attention variant trades some flexibility for lower computational cost by avoiding full self-attention over the history sequence.

The main systems advantage of the two-tower design lies in separability. User and post representations can be computed independently, which enables pre-inference post caching and approximate nearest-neighbor retrieval. This is a major reason two-tower architectures are attractive for media feed ranking systems [12]. In a deployment setting, the item tower can be run offline to produce a large table of candidate post embeddings, after which online inference only requires constructing the user embedding and querying the item index.

Limitations. The same separability that makes two-tower models efficient also constrains them. A pure dot-product interaction cannot expressively capture arbitrary cross-features between a specific user history and post. In addition, user cold-start remains difficult for our current two tower approach, as it relies heavily on content-based historical engagement rather than richer categorical profile or graph features [9].

3.3 FIT Two-Tower

Design additions. Our enhanced fully interacted two-tower model (FIT) augments the standard two-tower setup with a learnable Meta Query Module/Matrix (MQM), originally proposed in [13]. In this implementation, a candidate post embedding is first projected into a query space, scored against a learned bank of meta-query vectors, and then used to form either a soft combination of those queries during training or a hard-selected query at inference. This meta-query is then passed into the user encoder as an attention-pooling query, making the user representation candidate-aware earlier in the forward pass.

Increased early interaction: Conceptually, this changes the user encoder tower from a generic history encoder with late interaction (via dot-product) into an early interaction, item-conditioned history encoder:

$$p(y = 1 \mid \text{user}, \text{post}) = \sigma(\phi(\text{history}, q(\text{post}))^\top \mathbf{v}),$$

where $q(\text{post})$ is the MQM-derived meta-query and $\phi(\cdot)$ is the conditioned user encoder. In the current code, FIT can be used with either the full-transformer or cross-attention user encoder. The implementation maintains a learnable meta matrix and updates a temperature parameter τ for soft query selection during training.

The motivation for FIT is directly connected to the limitations of standard two-tower retrieval. In our FIT framework, a learnable meta matrix

$$Q \in \mathbb{R}^{N \times D}$$

stores implicit interaction patterns, and the candidate item embedding is used to form either a soft or hard query over this meta space [13], allowing the user-history encoder to emphasize different parts of a user’s prior behavior depending on the item being scored. Thus, the FIT model retains the majority of the systems structure of our standard two-tower architecture while introducing enhanced user–item interaction.

Lightweight similarity score (enhanced late interaction): Lightweight similarity scoring has been shown to outperform inexpressive dot-product scoring in terms of both effectiveness and efficiency [13]. In the current codebase, the optional *Lightweight Similarity Scorer (LSS)* is implemented in a simplified form, it applies a small MLP to the elementwise product of the user and item embeddings,

$$s(\text{user}, \text{post}) = \text{MLP}(\mathbf{u} \odot \mathbf{v}),$$

when the `fit_use_lss` option is enabled. Otherwise, FIT uses the standard dot product for scoring. Thus, in this implementation, the primary architectural gain comes from MQM-based early interaction, while late interaction can be modestly enhanced when simplified LSS is enabled.

Limitations. FIT increases interaction capacity but partially sacrifices the key advantage of standard two-tower retrieval: fully decoupled user and item representations. Once the user encoding depends on the candidate post, pre-inference post caching becomes less straightforward, and approximate nearest-neighbor retrieval becomes more complex, often requiring additional sharding and more advanced querying strategies.

3.4 MLP

Design. Our MLP is a pointwise binary classifier. It supports two user-history encodings; however, for the experiments here, we use a lightweight summarized encoding. With this summarized encoding, the user’s history is collapsed into a fixed-size vector using an exponential moving average (EMA). Because both the user summary and the target post embedding are 384-dimensional, the classifier input is a 768-dimensional concatenated vector [user summary || post embedding]. This vector is then passed through an MLP head consisting of a stack of fully connected blocks

$$\text{Linear} \rightarrow \text{BatchNorm1d} \rightarrow \text{GELU} \rightarrow \text{Dropout},$$

followed by a final linear layer and sigmoid output. In the runs reported here, the hidden-layer widths are [64, 32, 16].

In full-transformer mode, the MLP uses a learned transformer-based history encoder to compute a user vector from the entire prior-like sequence. In the code, this encoder first projects each 384-

dimensional history embedding through

$$\text{Linear} \rightarrow \text{LayerNorm} \rightarrow \text{GELU} \rightarrow \text{Dropout},$$

adds learned positional embeddings, applies a stack of transformer self-attention blocks, and then forms the user representation by combining learned-query attention pooling with masked mean pooling before an output projection. This learned user vector is then concatenated with the candidate post embedding and fed through the same MLP head described above. In both cases, the model outputs a scalar probability and is trained with binary cross-entropy on balanced positive/negative pairs.

Limitations. The main drawback is deployment cost. Because the MLP is a pointwise scorer over a concatenated user–post representation, it does not support efficient approximate nearest-neighbor retrieval in the way a standard two-tower model does. This means that although the MLP may achieve strong engagement-prediction accuracy, it is not the most practical model architecture for large-scale candidate generation.

3.5 Collaborative Filtering Baseline

Design. The collaborative-filtering baseline is an ID-only implicit-feedback logistic matrix-factorization model, closely aligned with the logistic MF formulation of [1]. Each training user and each training item receives a learned latent vector, together with user and item bias terms and a global bias:

$$s(u, i) = \mathbf{P}_u^\top \mathbf{Q}_i + a_u + b_i + g, \quad \hat{y}(u, i) = \sigma(s(u, i)).$$

Here, users are indexed from training-split `target.did` values, while items are indexed from the Stage 1 embedding IDs referenced by `like_emb_idx` and `neg_emb_idx`. The model does *not* use the stage 1, 384-dimensional post embeddings themselves. The latent factors are initialized with small Gaussian noise, while the bias terms are initialized at zero.

Loss. As in implicit-feedback logistic MF [1], observed user–item pairs are treated as positive interactions and all other pairs are treated as unobserved rather than explicit dislikes. The implementation optimizes a sampled approximation to the full implicit objective. Let B be a minibatch of observed positive pairs (u, i) , let r denote `negative_ratio`, and let w denote `negative_weight`. For each positive pair, the model samples r item IDs that are not in the user’s observed-positive set. The minibatch loss is

$$\mathcal{L}(B) = \frac{1}{|B|} \sum_{(u,i) \in B} c_{ui} \log(1 + e^{-s(u,i)}) + r w \frac{1}{|B|} \sum_{(u,i) \in B} \log(1 + e^{s(u,j)}),$$

where j denotes a sampled unobserved item for user u . Thus, the training script uses negative sampling rather than summing over the full set of missing entries as in the original full-matrix objective of [1].

Confidence weighting. In this simplified binary setting, the implicit interaction matrix is

$$r_{ui} \in \{0, 1\},$$

with $r_{ui} = 1$ if user u interacted with item i in the training data and $r_{ui} = 0$ otherwise. Following the weighted implicit-feedback view of logistic MF [1], all observed positive pairs receive the same confidence weight,

$$c_{ui} = 1 + \alpha,$$

where α is a tuning parameter.

Binary collaborative interaction matrix. Let R denote the binary user–item interaction matrix used for collaborative-filter training. Entries with $r_{ui} = 1$ correspond to observed user–item pairs, while entries with $r_{ui} = 0$ correspond to unobserved pairs.

Evaluation. Validation and holdout evaluation are performed on the shared candidate-pair datasets produced by earlier pipeline stages, where each target row contributes one positive candidate and one sampled negative candidate. The model score for a user–item pair (u, i) is the sigmoid-transformed logit

$$\hat{y}(u, i) = \sigma\left(\mathbf{P}_u^\top \mathbf{Q}_i + a_u + b_i + g\right),$$

and AUC and average precision are computed directly from these predicted probabilities. For hard classification metrics (e.g., confusion matrices), the script binarizes predicted probabilities using a configurable thresholding strategy, defaulting to the median user–item candidate score within each split, such that the top half of predictions are labeled as positive.

For unknown users or items, the script replaces the missing latent representation with the mean seen-user or mean seen-item representation. Inference-time user–item scoring is computed via the following cases:

1. Known user/item:

$$s(u, i) = \mathbf{P}_u^\top \mathbf{Q}_i + a_u + b_i + g$$

2. Unknown user:

$$s(\text{unk}, i) = \bar{\mathbf{P}}^\top \mathbf{Q}_i + \bar{a} + b_i + g$$

3. Unknown item:

$$s(u, \text{unk}) = \mathbf{P}_u^\top \bar{\mathbf{Q}} + a_u + \bar{b} + g$$

4. Unknown user/item:

$$s(\text{unk}, \text{unk}) = \bar{\mathbf{P}}^\top \bar{\mathbf{Q}} + \bar{a} + \bar{b} + g$$

where the bars denote means over seen users or items.

Limitations. This baseline inherits the standard limitations of ID-based collaborative filtering [1]. It does not use semantic post features and therefore cannot generalize through the expressive

384-dimensional content embeddings used by the other models. Cold-start remains particularly challenging for unseen users and items, since these cases collapse to shared mean fallback representations.

3.6 Experiments and Results

Hyperparameter sweeps were performed across all models during training and validation. Results from these sweeps are summarized below. The MLP baseline with a summarized mean user-history encoding achieved holdout AUC 0.8325 with dropout $p = 0.5$ and learning rate 1×10^{-3} . The standard two-tower model achieved 0.8314 holdout AUC with learning rate 3×10^{-4} and dropout 0.2. The best FIT two-tower model without lightweight similarity scoring (i.e., standard dot-product scoring) achieved holdout AUC **0.8339** with learning rate 5×10^{-4} and dropout 0.3. For the FIT two-tower model with lightweight similarity scoring (LSS), the best run achieved holdout AUC 0.8319 with a full-transformer encoder, $q = 64$, learning rate 5×10^{-4} , and dropout 0.3. The best collaborative-filtering model, using latent dimension 128 and $\alpha = 8$, achieved holdout AUC 0.4994 on the seen-user holdout under the corrected global median-rank thresholding rule, indicating near-random performance. A more robust collaborative-filtering sweep will likely be required as more positive and empirically observed negative user-item pairs are collected at deployment scale.

Recall, defined as $TP/(TP + FN)$, provides additional valuable insight into model capacity across architectures. The MLP baseline achieved holdout recall 0.7914, while the standard two-tower model achieved 0.8047. The best FIT two-tower model achieved holdout recall 0.7889, slightly below the standard two-tower despite its stronger candidate-conditioned user modeling. For the FIT two-tower model with LSS, the best run achieved the highest holdout recall 0.8108 under the same full-transformer, $q = 64$, learning rate 5×10^{-4} , and dropout 0.3 configuration. The best collaborative-filtering model achieved holdout recall 0.4981 on the seen-user holdout under a median-rank split, again indicating near-chance performance. Together, these recall results reinforce the AUC findings: content-based neural models substantially outperform collaborative filtering in this setting.

In addition to aggregate AUC comparisons, we analyze cold-start behavior by measuring recall as a function of the number of historical likes available for each user. Across all neural models (excluding collaborative filtering), recall exhibits a roughly logarithmic growth trend with respect to history length. However, the MLP shows the steepest drop in recall for users with fewer than three liked posts, suggesting greater sensitivity to sparse user histories and an increased tendency toward false negatives (as expected with randomly sampled negatives) [5].

To assess whether certain users are systematically favored by different models, we compute Gini coefficients over per-user precision, recall, F1, and AUC. Recall yields the most informative signal. The Gini coefficient provides a useful measure of how evenly performance is distributed across users and will be important for evaluating generalization under different modeling and regularization choices [17]. The MLP exhibits the highest inequality, with a Gini coefficient of 0.1684, compared to 0.1430 for the standard two-tower model (without enhanced early or late interaction).

Finally, we use Spearman correlation to assess what content the standard two-tower model promotes (via predicted engagement probability) relative to organic (pre-filtered) Bluesky user preferences. These analyses are preliminary, with visualizations provided by Will Schulz. Notably, the model shows a bias toward contemporary news and politics (see Figures 3.6 and 3.6). Additional observations suggest a tendency to promote more negative or toxic content, though not to an extreme degree (see Figures 3.6 and 3.6).

Table 1: Hyperparameter sweep results for the MLP baseline, sorted by holdout AUC.

Learning rate	Dropout	Validation AUC	Holdout AUC
0.0010	0.5	0.8521	0.8325
0.0003	0.5	0.8516	0.8324
0.0003	0.3	0.8513	0.8321
0.0010	0.3	0.8519	0.8321

Table 2: Hyperparameter sweep results for two-tower models without FIT, sorted by holdout AUC.

User encoder	Learning rate	Dropout	Validation AUC	Holdout AUC
full_transformer	0.0003	0.2	0.8524	0.8314
cross_attention	0.0010	0.2	0.8489	0.8284
cross_attention	0.0003	0.2	0.8508	0.8282
full_transformer	0.0010	0.2	0.8486	0.8280
full_transformer	0.0010	0.1	0.8473	0.8269
full_transformer	0.0003	0.1	0.8463	0.8250
cross_attention	0.0010	0.1	0.8437	0.8229
cross_attention	0.0003	0.1	0.8423	0.8190

User encoder	Learning rate	Dropout	Validation AUC	Holdout AUC
full_transformer (q=64)	0.0005	0.3	0.8549	0.8339
full_transformer (q=64)	0.0001	0.3	0.8545	0.8331
full_transformer (q=128)	0.0005	0.3	0.8524	0.8329
full_transformer (q=64)	0.0003	0.3	0.8527	0.8322
full_transformer (q=128)	0.0005	0.2	0.8521	0.8320
full_transformer (q=64)	0.0003	0.2	0.8525	0.8312
full_transformer (q=64)	0.0001	0.2	0.8519	0.8300
cross_attention (q=64)	0.0005	0.3	0.8506	0.8296
full_transformer (q=64)	0.0005	0.2	0.8509	0.8296
full_transformer (q=128)	0.0001	0.3	0.8499	0.8294
full_transformer (q=128)	0.0001	0.2	0.8504	0.8290
full_transformer (q=128)	0.0003	0.3	0.8499	0.8289
cross_attention (q=64)	0.0005	0.2	0.8491	0.8281
full_transformer (q=128)	0.0003	0.2	0.8493	0.8275
cross_attention (q=64)	0.0003	0.3	0.8494	0.8274
cross_attention (q=64)	0.0003	0.2	0.8449	0.8245
cross_attention (q=64)	0.0001	0.2	0.8440	0.8233
cross_attention (q=64)	0.0001	0.3	0.8431	0.8226
cross_attention (q=128)	0.0005	0.3	0.8418	0.8214
cross_attention (q=128)	0.0005	0.2	0.8408	0.8205
cross_attention (q=128)	0.0001	0.2	0.8384	0.8174
cross_attention (q=128)	0.0003	0.3	0.8390	0.8170
cross_attention (q=128)	0.0003	0.2	0.8390	0.8169
cross_attention (q=128)	0.0001	0.3	0.8358	0.8142

Table 3: Hyperparameter sweep results for FIT two-tower models, sorted by holdout AUC. MQM module query count q is shown in the user-encoder column.

User encoder	Learning rate	Dropout	Validation AUC	Holdout AUC
CF (128, $\alpha = 8$)	0.0003	—	0.4930	0.4994
CF (64, $\alpha = 8$)	0.0003	—	0.4896	0.4993
CF (64, $\alpha = 32$)	0.0003	—	0.4876	0.4992
CF (64, $\alpha = 8$)	0.0010	—	0.4946	0.4992
CF (128, $\alpha = 32$)	0.0003	—	0.4918	0.4991
CF (64, $\alpha = 32$)	0.0010	—	0.4930	0.4987
CF (128, $\alpha = 8$)	0.0010	—	0.4881	0.4980
CF (128, $\alpha = 32$)	0.0010	—	0.4853	0.4974
CF (256, $\alpha = 8$)	0.0010	—	0.4889	0.4957
CF (256, $\alpha = 8$)	0.0003	—	0.4937	0.4957
CF (256, $\alpha = 32$)	0.0003	—	0.4928	0.4955
CF (256, $\alpha = 32$)	0.0010	—	0.4865	0.4952

Table 4: Hyperparameter sweep results for Collaborative-filtering models over latent dimension, learning rate, and positive-confidence scaling on the seen-user holdout, sorted by holdout AUC.

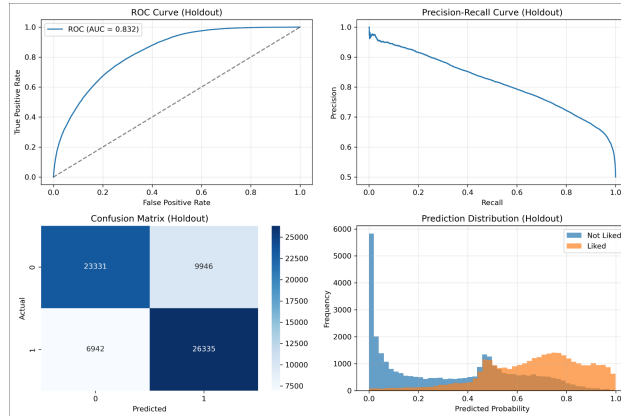


Figure 2: Holdout performance: ROC and precision–recall curves, confusion matrix, and predicted probability distributions for the MLP model.

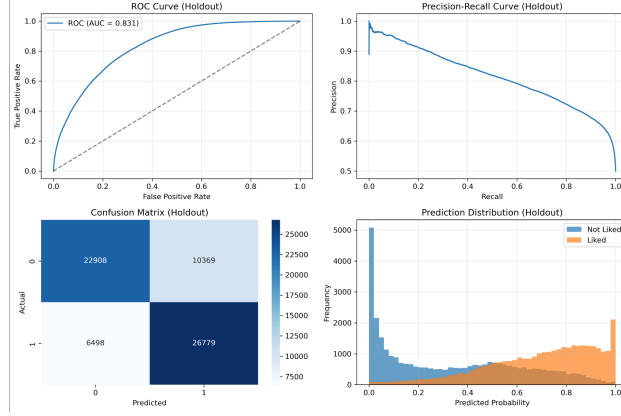


Figure 3: Holdout performance: ROC and precision–recall curves, confusion matrix, and predicted probability distributions for the two-tower model (no fit).

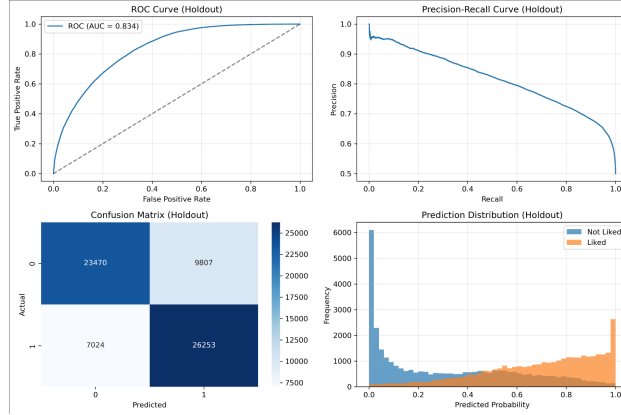


Figure 4: Holdout performance: ROC and precision–recall curves, confusion matrix, and predicted probability distributions for the two-tower model (with fit).

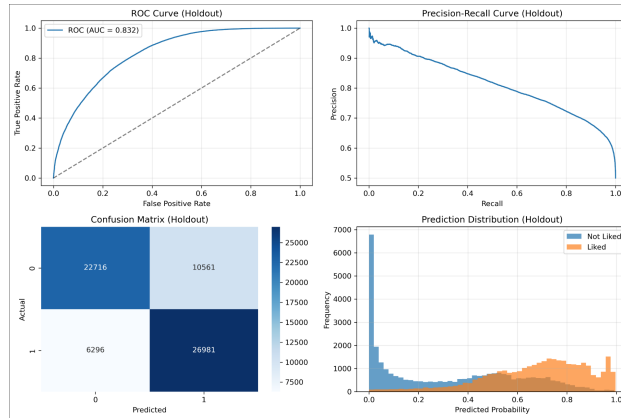


Figure 5: Holdout performance for the two-tower model with fit and LSS, including ROC, precision–recall, confusion matrix, and prediction distribution.

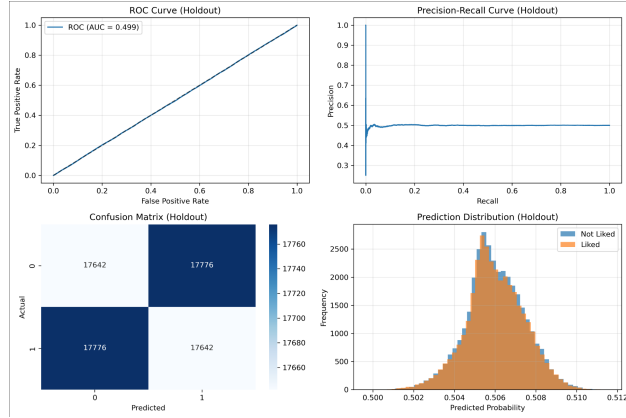


Figure 6: Holdout performance: ROC and precision–recall curves, confusion matrix, and predicted probability distributions for the collaborative filtering model.

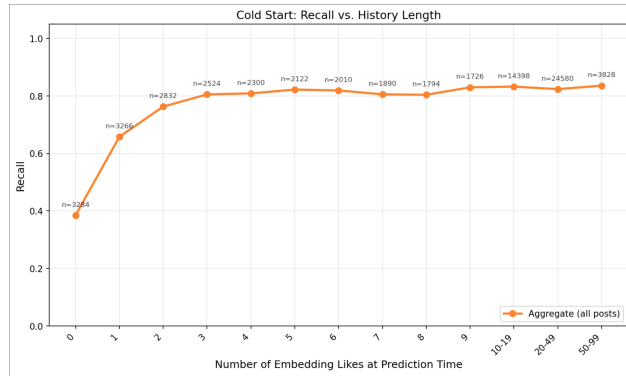


Figure 7: Cold-start analysis: recall vs. user history length for the MLP model.

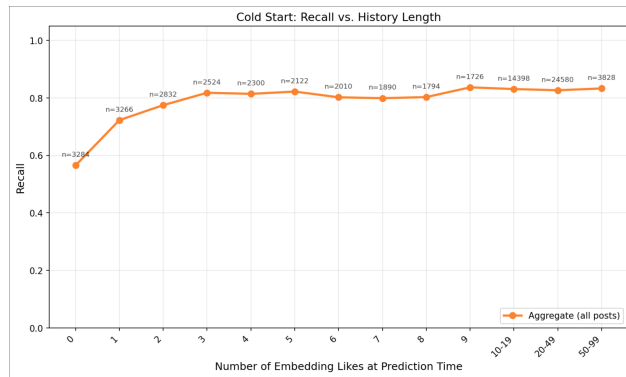


Figure 8: Cold-start analysis: recall vs. user history length for the two-tower model (no fit).

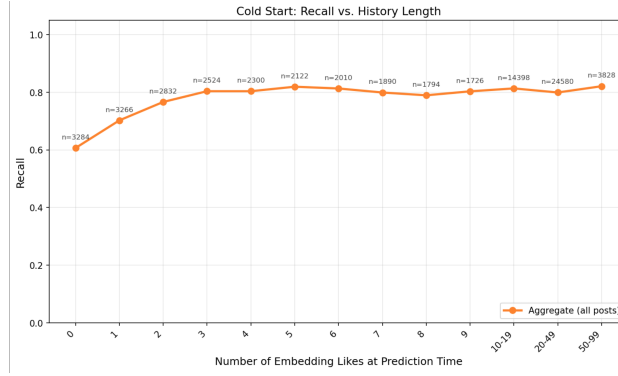


Figure 9: Cold-start analysis: recall vs. user history length for the two-tower model (with fit).

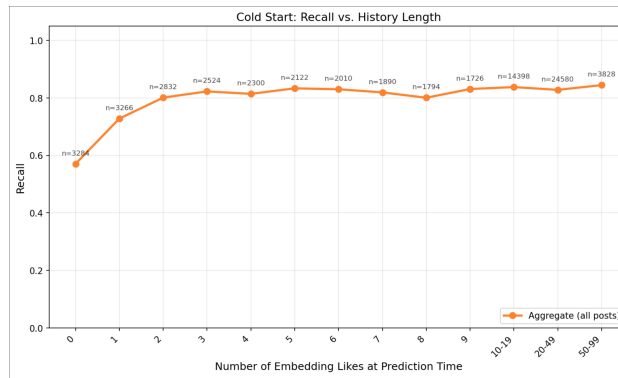


Figure 10: Cold-start analysis: recall vs. user history length for the two-tower model with fit and LSS.

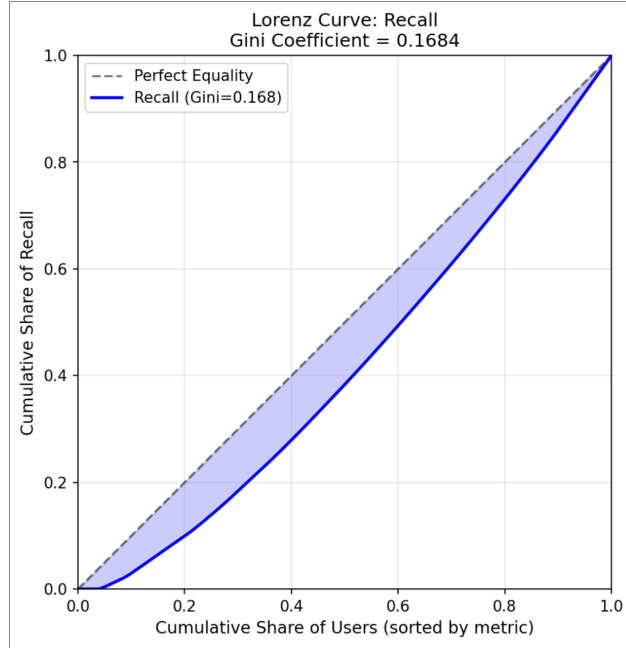


Figure 11: Lorenz curve of recall distribution across users, with Gini coefficient, for the MLP model.

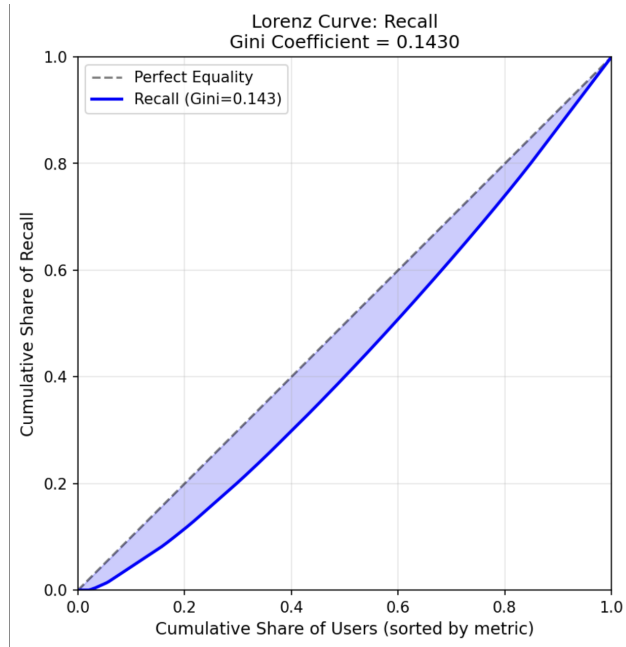


Figure 12: Lorenz curve of recall distribution across users, with Gini coefficient, for the two-tower model (no fit).

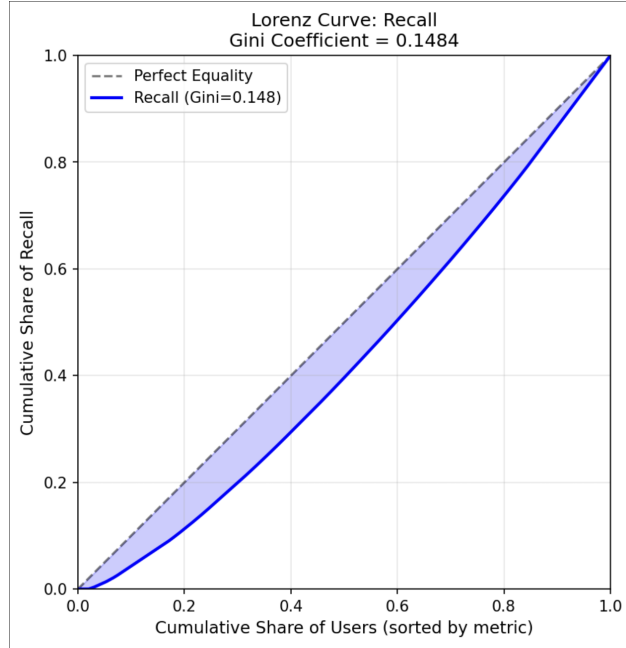


Figure 13: Lorenz curve of recall distribution across users, with Gini coefficient, for the two-tower model (with fit).

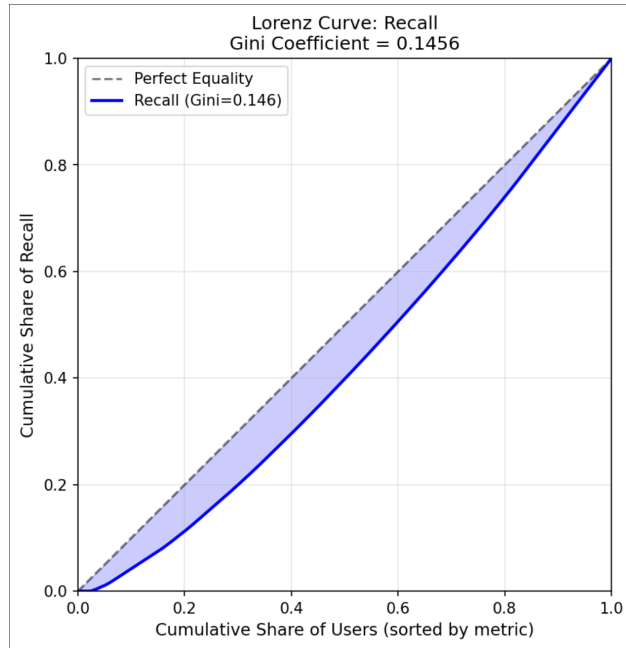
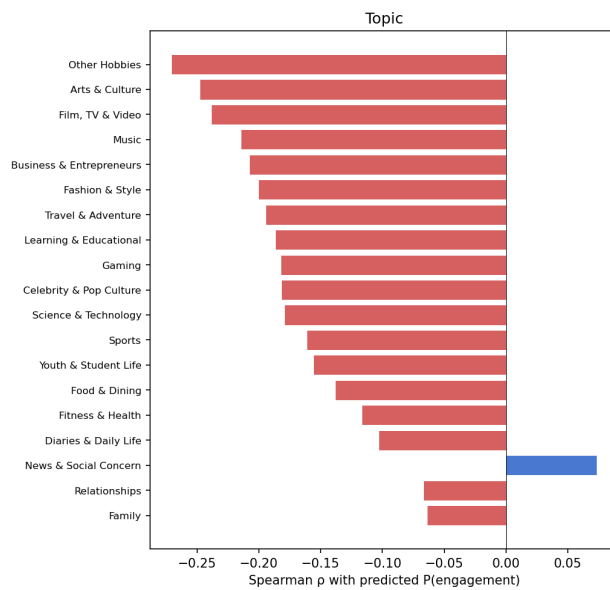
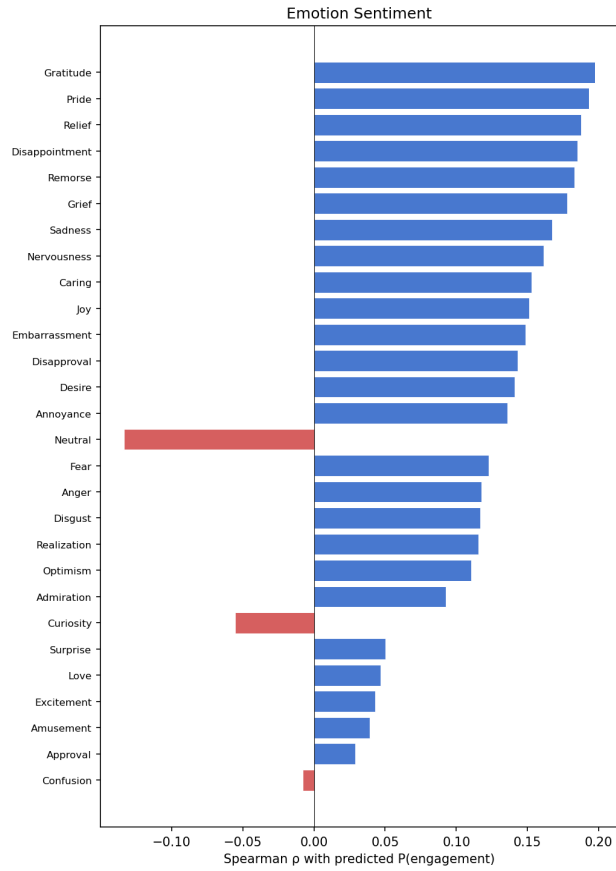
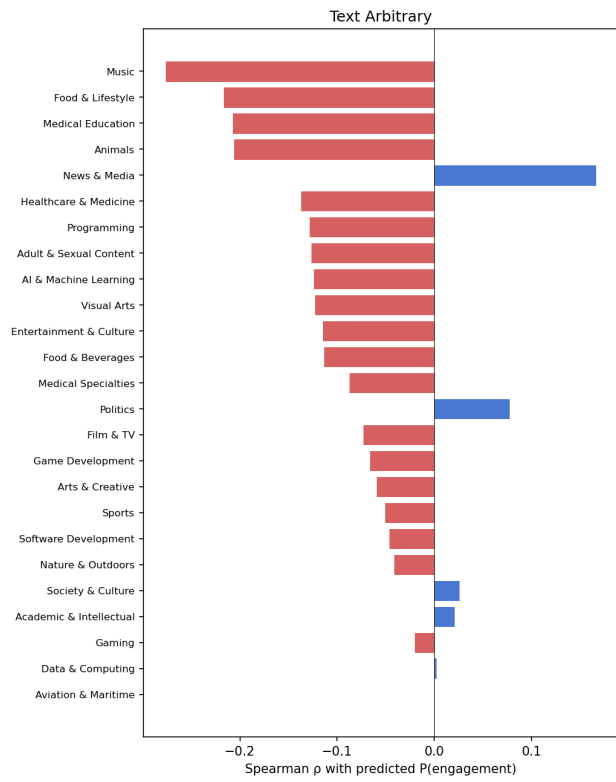
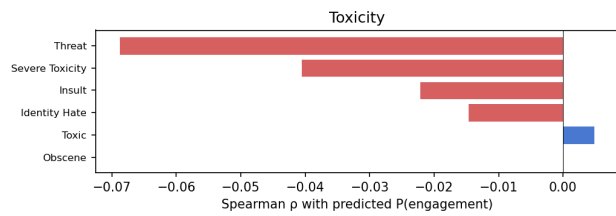
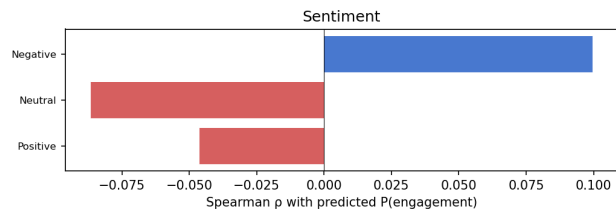


Figure 14: Lorenz curve of recall distribution across users, with Gini coefficient, for the two-tower model with fit and LSS.

The following preliminary qualitative analysis figures for the two-tower model's predictions were provided by Will Schulz, Principal Investigator overseeing this research:





4 Analysis

Interpreting confusion matrices: While AUC is the primary evaluation metric used here, precision and F1 will become more informative once true negative user-item interaction pairs are available. In the current setting, recall is the most informative metric, as it directly captures our ability to recover known positive interactions. Formally, recall measures the proportion of true positives correctly identified.

Interpreting results for the MLP model: The MLP performs slightly better than standard FIT in terms of AUC, indicating that it benefits from modeling full user-item interactions during training. However, it likely underperforms the FIT model without LSS in AUC due to misleading signals from randomly sampled negative examples. With its greater interaction capacity, the MLP likely overfits to these sampled negatives, adjusting its weights to better fit potentially misleading signals. This creates a trade-off between correctly classifying positive and negative user-item pairs, similar to what is observed between the MLP and FIT (no LSS) models.

Standard two-tower (no enhanced interaction): The standard two-tower model performs worst in terms of AUC. With limited interaction capacity, the model is less able to capture fine-grained user-item feature relationships. This is reflected in the confusion matrix, where it produces more positive predictions overall, increasing both true positives and false positives.

Two-tower with/without FIT: The FIT two-tower model without LSS effectively distinguishes between positive and negative user-item pairs. However, its limited late interaction (dot product) restricts its ability to capture nuanced feature interactions. As a result, it achieves strong AUC but can be overconfident in its predictions, even when many negatives correspond to items the user has simply not yet been exposed to.

The FIT two-tower model with enhanced early and late interaction (LSS instead of a dot product) produces a probability distribution more similar to the MLP. This is very promising, as it achieves the highest recall by a clear margin while better capturing signal from both positive and sampled-negative pairs. In particular, it appears to identify items that users may have liked if properly exposed. Additionally, the FIT model with LSS achieves a lower Gini coefficient, suggesting improved generalization across users rather than concentrating performance on a subset of (likely highly active) users.

Limitations of collaborative filtering: Cold-start issues are a well-known challenge in recommendation systems [16]. In our setting, this arises when predicting for new users or items with limited historical data. The collaborative filtering (CF) model performs poorly (near-random), highlighting its sensitivity to our cold-start constraints and a mismatch with our prediction setting. Due to temporal splitting and rapid content turnover, many holdout items are unseen during training. Performance is further degraded by sparse user-item interactions after filtering, as limited co-like behavior weakens the collaborative signal.

While the CF baseline remains conceptually important, sparse and rapidly evolving interactions limit its (collaborative) effectiveness, as users often lack overlapping exposure to the same items. Additionally, the unseen-user holdout setting provides little to no personalization signal for CF,

whereas neural models can still leverage content features and partial histories. As such, CF is best viewed here as a diagnostic baseline on seen-user holdout, rather than a deployment-ready model, though its performance may improve with revised data splits and architecture better aligned to the feed-ranking setting.

Limitations of current evaluation without top- k ANN feed generation: While the mean summarized-history MLP achieves strong AUC and accuracy under limited hyperparameter sweeps, this does not imply suitability for feed ranking. MLPs lack the decoupled embedding structure required for efficient ANN-based candidate generation. Therefore, MLP results should be interpreted as strong comparison baselines against decoupled user-item tower models, rather than as evidence of the model architecture’s suitability for inference-time feed ranking deployment.

5 Conclusion

The results of our modular engagement-prediction pipeline thus far demonstrate that a two-tower model with enhanced early and late interaction (FIT) can predict positive engagement at a level comparable to an MLP with full user-post interaction. The FIT model with LSS scoring achieves the highest recall at 0.8108 while also capturing signal from confounding negative user-item pairs that may have been positive with proper exposure. Additionally, analysis of recall Gini inequality across holdout users shows the superior ability of the FIT two-tower model with LSS to generalize and make accurate predictions across the entire user base.

These results, along with the poor performance of collaborative filtering in this domain, highlight the viability of enhanced early-late interaction two-tower architectures for social media engagement prediction. In summary, FIT with LSS achieves a prediction distribution similar to the MLP while maintaining better generalizability across users and preserving the efficiency advantages of a decoupled two-tower architecture.

5.1 Future Work

First, approximate nearest-neighbor (ANN) retrieval should be implemented for the two-tower pipeline at inference time to enable the generation of mock user feeds. State-of-the-art locality-sensitive hashing methods such as FastLSH and DB-LSH provide useful starting points for scalable top- k retrieval experiments [6, 7]. In particular, DB-LSH improves candidate retrieval through dynamic, query-centric bucketing, while FastLSH reduces the computational cost of hash evaluation, suggesting that the two approaches may be complementary in a large-scale retrieval stack. Ultimately, generating mock user feeds would enable valuable qualitative evaluation beyond the current pairwise setup.

Second, to improve the effectiveness of our positive/negative sampling approach, we need to collect additional data on posts that users were exposed to but did not engage with. This will become more feasible as we move beyond Bluesky-provided data and instead leverage data from our own Skylight deployment feeds.

References

- [1] Christopher C. Johnson. Logistic matrix factorization for implicit feedback data. Presented at the NIPS 2014 Workshop on Distributed Machine Learning and Matrix Computations, 2014. URL <https://web.stanford.edu/~rezab/nips2014workshop/submits/logmat.pdf>. Spotify, New York, NY. Available online as workshop paper.
- [2] Yehuda Koren, Robert Bell, and Chris Volinsky. Matrix factorization techniques for recommender systems. *Computer*, 42(8):30–37, 2009. doi: 10.1109/MC.2009.263.
- [3] Meta. How instagram feed ranking works, 2023. URL <https://transparency.meta.com/features/explaining-ranking/ig-feed/>. Accessed: 2026-03-18.
- [4] M Viswa Murali, T G Vishnu, and Nancy Victor. A collaborative filtering based recommender system for suggesting new trends in any domain of research. In *2019 5th International Conference on Advanced Computing Communication Systems (ICACCS)*, pages 550–553, 2019. doi: 10.1109/ICACCS.2019.8728409.
- [5] Olli Rainio. Evaluation metrics and statistical tests for machine learning. *Journal of Biomedical Informatics*, 2024.
- [6] Zongyuan Tan, Hongya Wang, Bo Xu, Minjie Luo, and Ming Du. Fast locality sensitive hashing with theoretical guarantee, 2023. URL <https://arxiv.org/abs/2309.15479>.
- [7] Yao Tian, Xi Zhao, and Xiaofang Zhou. Db-lsh: Locality-sensitive hashing with query-based dynamic bucketing, 2022. URL <https://arxiv.org/abs/2207.07823>.
- [8] Zeynep Tufekci. The algorithm giveth but it also taketh away. *Medium*, May 2016. URL <https://medium.com/message/the-algorithm-giveth-but-it-also-taketh-b7efad92bc1f>.
- [9] Shoujin Wang, Liang Hu, Yan Wang, Xiangnan He, Quan Z. Sheng, Mehmet A. Orgun, Longbing Cao, Francesco Ricci, and Philip S. Yu. Graph learning based recommender systems: A review, 2021. URL <https://arxiv.org/abs/2105.06339>.
- [10] Yihan Wang, Fei Xiong, Zhexin Han, Qi Song, Kaiqiao Zhan, and Ben Wang. Unleashing the potential of two-tower models: Diffusion-based cross-interaction for large-scale matching, 2025. URL <https://arxiv.org/abs/2502.20687>.
- [11] Candace L. White and Brandon Boatwright. Social media ethics in the data economy: Issues of social responsibility for using facebook for public relations. *Public Relations Review*, 46(5):101980, 2020. ISSN 0363-8111. doi: 10.1016/j.pubrev.2020.101980. URL <https://www.sciencedirect.com/science/article/pii/S0363811120301077>.
- [12] xAI. X algorithm: Recommendation system for the "for you" feed. <https://github.com/xai-org/x-algorithm>, 2026.

- [13] Chao Xiong, Xianwen Yu, Wei Xu, Lei Cheng, Chuan Yuan, and Linjian Mo. A learnable fully interacted two-tower model for pre-ranking system. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval*, SIGIR '25, pages 2182–2191. ACM, July 2025. doi: 10.1145/3726302.3729881. URL <https://doi.org/10.1145/3726302.3729881>.
- [14] YouTube. How youtube works: Recommendations, 2023. URL <https://www.youtube.com/howyoutubeworks/recommendations/>. Accessed: 2026-03-18.
- [15] Yantao Yu. A dual augmented two-tower model for online large-scale recommendation. 2021. URL <https://api.semanticscholar.org/CorpusID:252924234>.
- [16] Weizhi Zhang, Yuanchen Bei, Liangwei Yang, Henry Peng Zou, Peilin Zhou, Aiwei Liu, Yinghui Li, Hao Chen, Jianling Wang, Yu Wang, Feiran Huang, Sheng Zhou, Jiajun Bu, Allen Lin, James Caverlee, Fakhri Karray, Irwin King, and Philip S. Yu. Cold-start recommendation towards the era of large language models (llms): A comprehensive survey and roadmap, 2025. URL <https://arxiv.org/abs/2501.01945>.
- [17] Y. Zhang, B. Nie, J. Du, J. Chen, Y. Du, H. Jin, X. Zheng, X. Chen, and Z. Miao. Feature selection based on neighborhood rough sets and gini index. *PeerJ Computer Science*, 9:e1711, 2023. doi: 10.7717/peerj-cs.1711.